State space models I

This version: October 29, 2026

ECMA 31150, Spring 2026

University of Chicago

Department of Economics

Mikkel Plagborg-Moller

Plan for this slide deck

1. Modeling latent variables with state space models
2. Inferring the latent variables
3. Bayesian inference on state space models
4. Application: GDP measurement

Models with latent variables

- In many settings, we'd like to consider models with **latent** (i.e., unobserved) variables.
- Our observed data is related to the latent variables, but with added noise.
- **State space models** form a rich class of models with latent variables.
- As we'll see in the next slide deck, state space models also facilitate analysis of time-varying parameters, missing data, and mixed-frequency data.

Linear-Gaussian state space model

- **Observation equation** for $n \times 1$ observed data $\boldsymbol{y}_t$:

$$\boldsymbol{y}_t = \boldsymbol{c} + \boldsymbol{G}\boldsymbol{s}_t + \boldsymbol{v}_t, \quad \boldsymbol{v}_t \overset{iid}{\sim} N(\boldsymbol{0}, \boldsymbol{V}).$$

  - $\boldsymbol{v}_t$: measurement error.

- **Transition equation** for $m \times 1$ latent **state** vector $\boldsymbol{s}_t$:

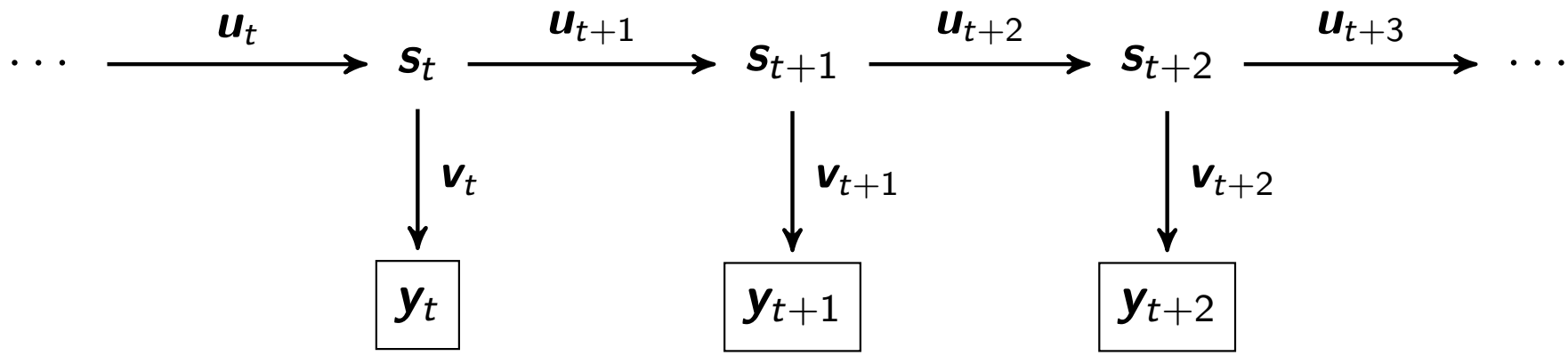
$$\boldsymbol{s}_t = \boldsymbol{F}\boldsymbol{s}_{t-1} + \boldsymbol{u}_t, \quad \boldsymbol{u}_t \overset{iid}{\sim} N(\boldsymbol{0}, \boldsymbol{U}).$$

  - $\boldsymbol{u}_t$: shocks to the state vector.

- Initial condition:

$$\boldsymbol{s}_0 \sim N(\boldsymbol{s}_{0|0}, \boldsymbol{P}_{0|0}).$$

- $\boldsymbol{s}_0$, $\{\boldsymbol{u}_t\}$, and $\{\boldsymbol{v}_t\}$ are all mutually independent.


$\cdots$
u_t
s_t
u_{t+1}
s_{t+1}
u_{t+2}
s_{t+2}
u_{t+3}
$\cdots$
v_t
v_{t+1}
v_{t+2}
y_t
y_{t+1}
y_{t+2}

Example 1: dynamic factor model

- Consider a model where n observed series $y_{1t}, \ldots, y_{nt}$ are driven by a latent **factor** f_t:

$$y_{it} = c_i + \lambda_i f_t + v_{it}, \quad v_{it} \overset{iid}{\sim} N(0, \sigma_i^2).$$

- f_t could be thought of as the overall state of the business cycle.
- The idiosyncratic errors v_{it} are assumed independent across i. So f_t is responsible for the co-movement between the observed series.
- Assume f_t evolves as an AR(p) process:

$$f_t = a_1 f_{t-1} + \cdots + a_p f_{t-p} + u_t, \quad u_t \overset{iid}{\sim} N(0, 1).$$

- We need to normalize either the scale of the **loadings** λ_i or the factor f_t. Here set $\text{Var}(u_t) = 1$.

Example 1: dynamic factor model (cont.)

- Written in state space form:

$$\boldsymbol{y}_t = \boldsymbol{c} + \boldsymbol{G}\boldsymbol{s}_t + \boldsymbol{v}_t, \quad \boldsymbol{v}_t \overset{iid}{\sim} N(\boldsymbol{0}, \boldsymbol{V}),$$

$$\boldsymbol{s}_t = \boldsymbol{F}\boldsymbol{s}_{t-1} + \boldsymbol{u}_t, \quad \boldsymbol{u}_t \overset{iid}{\sim} N(\boldsymbol{0}, \boldsymbol{U}),$$

where

$$\boldsymbol{c} \equiv \begin{pmatrix} c_1 \\ \vdots \\ c_n \end{pmatrix}, \ \boldsymbol{G} \equiv \begin{pmatrix} \lambda_1 & 0 & \dots & 0 \\ \vdots & \vdots & \vdots & \vdots \\ \lambda_n & 0 & \dots & 0 \end{pmatrix}, \ \boldsymbol{V} \equiv \begin{pmatrix} \sigma_1^2 & & & \\ & \ddots & & \\ & & & \sigma_n^2 \end{pmatrix},$$

$$\boldsymbol{s}_t \equiv \begin{pmatrix} f_t \\ f_{t-1} \\ \vdots \\ f_{t-p+1} \end{pmatrix}, \ \boldsymbol{F} \equiv \begin{pmatrix} a_1 & a_2 & \cdots & a_p \\ 1 & & & 0 \\ & \ddots & & \vdots \\ & & 1 & 0 \end{pmatrix}, \ \boldsymbol{U} \equiv \begin{pmatrix} 1 & & & \\ & 0 & & \\ & & \ddots & \\ & & & 0 \end{pmatrix}.$$

Example 1: dynamic factor model (cont.)

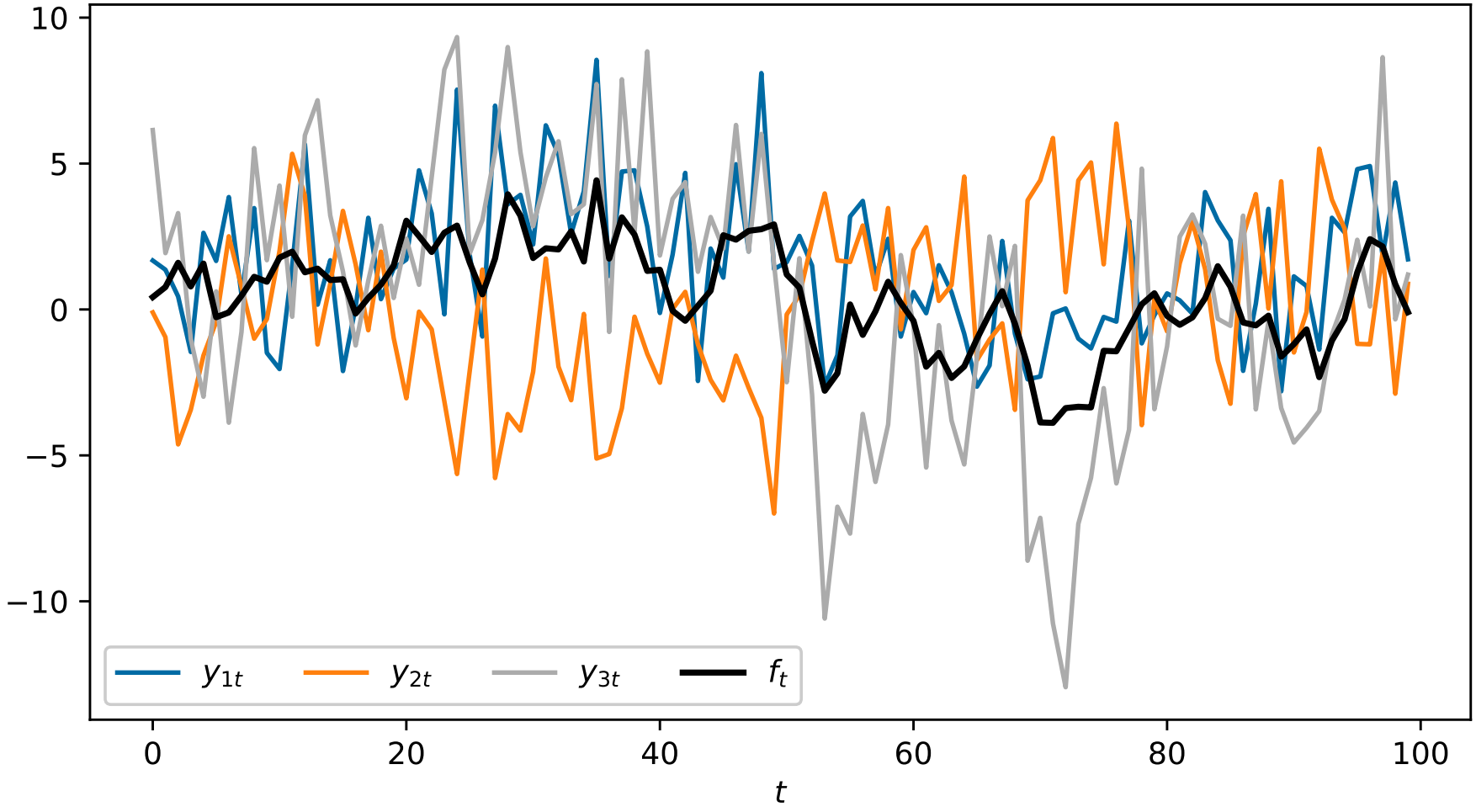
- Let's simulate data from the factor model:

$$y_{it} = c_i + \lambda_i f_t + v_{it}, \quad v_{it} \overset{iid}{\sim} N(0, \sigma_i^2),$$

$$f_t = a_1 f_{t-1} + u_t, \quad u_t \overset{iid}{\sim} N(0, 1).$$

- $n = 3$ observed variables y_{1t}, y_{2t}, y_{3t}.
- True parameters:

$$\begin{pmatrix} c_1 \\ c_2 \\ c_3 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad \begin{pmatrix} \lambda_1 \\ \lambda_2 \\ \lambda_3 \end{pmatrix} = \begin{pmatrix} 1 \\ -1 \\ 2 \end{pmatrix}, \quad \begin{pmatrix} \sigma_1^2 \\ \sigma_2^2 \\ \sigma_3^2 \end{pmatrix} = \begin{pmatrix} 4 \\ 4 \\ 9 \end{pmatrix}.$$


10
5
0
−5
−10
0
20
40
60
80
100
t
y_{1t}
y_{2t}
y_{3t}
f_t

Example 2: stationary ARMA(1, 1)

$$y_t = \rho y_{t-1} + u_t + \theta u_{t-1}, \quad u_t \overset{iid}{\sim} N(0, \sigma^2)$$

$$\Downarrow$$

$$y_t = (1 - \rho L)^{-1}(1 + \theta L)u_t = (1 + \theta L)x_t,$$

where $x_t \equiv (1 - \rho L)^{-1} u_t$ satisfies $x_t = \rho x_{t-1} + u_t$

$$\Downarrow$$

$$y_t = \boldsymbol{G}\boldsymbol{s}_t + v_t, \quad v_t \overset{iid}{\sim} N(0, 0),$$

$$\boldsymbol{s}_t = \boldsymbol{F}\boldsymbol{s}_{t-1} + \boldsymbol{u}_t, \quad \boldsymbol{u}_t \overset{iid}{\sim} N(\boldsymbol{0}, \boldsymbol{U}),$$

where $\boldsymbol{s}_t \equiv \begin{pmatrix} x_t \\ x_{t-1} \end{pmatrix}$, $\boldsymbol{G} \equiv \begin{pmatrix} 1 & \theta \end{pmatrix}$, $\boldsymbol{F} \equiv \begin{pmatrix} \rho & 0 \\ 1 & 0 \end{pmatrix}$, $\boldsymbol{U} \equiv \begin{pmatrix} \sigma^2 & 0 \\ 0 & 0 \end{pmatrix}$.

- General ARMA(p, q): Brockwell & Davis textbook, Example 9.3.2.

Inferring the latent states

- How can we infer the latent states $\{\boldsymbol{s}_t\}$ from the observed data $\{\boldsymbol{y}_t\}$?
- Assume for now that we know the true parameters of the state space model (more on estimation later).
- Since both states and data are Gaussian, we're working with conditional distributions of Gaussian random vectors.
- We just need to keep track of conditional means and variances:

$$\boldsymbol{s}_{t|\tau} \equiv E[\boldsymbol{s}_t \mid \{\boldsymbol{y}_s\}_{1\le s\le \tau}], \quad \boldsymbol{P}_{t|\tau} \equiv \mathrm{Var}[\boldsymbol{s}_t \mid \{\boldsymbol{y}_s\}_{1\le s\le \tau}],$$

$$\boldsymbol{y}_{t|\tau} \equiv E[\boldsymbol{y}_t \mid \{\boldsymbol{y}_s\}_{1\le s\le \tau}], \quad \boldsymbol{Q}_{t|\tau} \equiv \mathrm{Var}[\boldsymbol{y}_t \mid \{\boldsymbol{y}_s\}_{1\le s\le \tau}].$$

Inferring the latent states (cont.)

- We will proceed inductively/iteratively. Take as given the information $\{\boldsymbol{s}_{t-1|t-1}, \boldsymbol{P}_{t-1|t-1}\}$ about $\boldsymbol{s}_{t-1}$ using data up to time $t-1$.
- Then (exercise: verify)

$$\boldsymbol{s}_{t|t-1} = E[\boldsymbol{F}\boldsymbol{s}_{t-1} + \boldsymbol{u}_t \mid \{\boldsymbol{y}_s\}_{s\leq t-1}] = \boxed{\boldsymbol{F}\boldsymbol{s}_{t-1|t-1}},$$

$$\boldsymbol{P}_{t|t-1} = \mathrm{Var}(\boldsymbol{F}\boldsymbol{s}_{t-1} + \boldsymbol{u}_t \mid \{\boldsymbol{y}_s\}_{s\leq t-1}) = \boxed{\boldsymbol{F}\boldsymbol{P}_{t-1|t-1}\boldsymbol{F}' + \boldsymbol{U}},$$

$$\boldsymbol{y}_{t|t-1} = E[\boldsymbol{c} + \boldsymbol{G}\boldsymbol{s}_t + \boldsymbol{v}_t \mid \{\boldsymbol{y}_s\}_{s\leq t-1}] = \boxed{\boldsymbol{c} + \boldsymbol{G}\boldsymbol{s}_{t|t-1}},$$

$$\boldsymbol{Q}_{t|t-1} = \mathrm{Var}(\boldsymbol{c} + \boldsymbol{G}\boldsymbol{s}_t + \boldsymbol{v}_t \mid \{\boldsymbol{y}_s\}_{s\leq t-1}) = \boxed{\boldsymbol{G}\boldsymbol{P}_{t|t-1}\boldsymbol{G}' + \boldsymbol{V}}.$$

- These formulas tell us how to forecast $\boldsymbol{s}_t$ and $\boldsymbol{y}_t$ one step ahead.

Inferring the latent states (cont.)

- $\{\boldsymbol{s}_t, \boldsymbol{y}_t\}$ are jointly Gaussian given $\{\boldsymbol{y}_s\}_{s \le t-1}$.
- By the standard formula for the conditional distribution of jointly Gaussian vectors (Brockwell & Davis textbook, Proposition A.3.1):

$$\boldsymbol{s}_{t|t} = E[\boldsymbol{s}_t \mid \{\boldsymbol{y}_s\}_{s \le t}] = \boxed{\boldsymbol{s}_{t|t-1} + \boldsymbol{K}_t(\boldsymbol{y}_t - \boldsymbol{y}_{t|t-1})},$$

$$\text{where} \quad \boldsymbol{K}_t \equiv \text{Cov}(\boldsymbol{s}_t, \boldsymbol{y}_t \mid \{\boldsymbol{y}_s\}_{s \le t-1}) \boldsymbol{Q}_{t|t-1}^{-1}.$$

- Moreover, by that same formula,

$$\begin{aligned} \boldsymbol{P}_{t|t} &= \text{Var}(\boldsymbol{s}_t \mid \{\boldsymbol{y}_s\}_{s \le t}) \\ &= \boxed{\boldsymbol{P}_{t|t-1} - \text{Cov}(\boldsymbol{s}_t, \boldsymbol{y}_t \mid \{\boldsymbol{y}_s\}_{s \le t-1}) \boldsymbol{Q}_{t|t-1}^{-1} \text{Cov}(\boldsymbol{y}_t, \boldsymbol{s}_t \mid \{\boldsymbol{y}_s\}_{s \le t-1})}. \end{aligned}$$

- Exercise: use the state space model equations to show

$$\text{Cov}(\boldsymbol{s}_t, \boldsymbol{y}_t \mid \{\boldsymbol{y}_s\}_{s \le t-1}) = \boldsymbol{P}_{t|t-1} \boldsymbol{G}'.$$

Kalman filter

- The preceding derivations yield the **Kalman filter** recursions:

1. Initialize $\{\boldsymbol{s}_{0|0}, \boldsymbol{P}_{0|0}\}$.
2. For each $t = 1, 2, \ldots, T$:

   a) *Predict:*

$$\begin{aligned}
\boldsymbol{s}_{t|t-1} &= \boldsymbol{F}\boldsymbol{s}_{t-1|t-1}, \\
\boldsymbol{P}_{t|t-1} &= \boldsymbol{F}\boldsymbol{P}_{t-1|t-1}\boldsymbol{F}' + \boldsymbol{U}, \\
\boldsymbol{y}_{t|t-1} &= \boldsymbol{c} + \boldsymbol{G}\boldsymbol{s}_{t|t-1}, \\
\boldsymbol{Q}_{t|t-1} &= \boldsymbol{G}\boldsymbol{P}_{t|t-1}\boldsymbol{G}' + \boldsymbol{V}.
\end{aligned}$$

   b) **Kalman gain**: $\boldsymbol{K}_t = \boldsymbol{P}_{t|t-1}\boldsymbol{G}'\boldsymbol{Q}_{t|t-1}^{-1}$.

   c) *Update:*

$$\begin{aligned}
\boldsymbol{s}_{t|t} &= \boldsymbol{s}_{t|t-1} + \boldsymbol{K}_t(\boldsymbol{y}_t - \boldsymbol{y}_{t|t-1}), \\
\boldsymbol{P}_{t|t} &= \boldsymbol{P}_{t|t-1} - \boldsymbol{K}_t\boldsymbol{Q}_{t|t-1}\boldsymbol{K}_t'.
\end{aligned}$$

Kalman filter: initialization

- We need to pick a "prior belief" about the initial state $\boldsymbol{s}_0$:

$$\boldsymbol{s}_0 \sim N(\boldsymbol{s}_{0|0}, \boldsymbol{P}_{0|0}).$$

- If $\{\boldsymbol{s}_t\}$ is stationary, it makes sense to initialize at the stationary distribution:

$$\boldsymbol{s}_{0|0} = \boldsymbol{0}, \quad \boldsymbol{P}_{0|0} = \text{Var}(\boldsymbol{s}_t),$$

  where (as we know) the latter solves the Lyapunov equation

$$\text{Var}(\boldsymbol{s}_t) = \boldsymbol{F}\,\text{Var}(\boldsymbol{s}_t)\boldsymbol{F}' + \boldsymbol{U}.$$

- If $\{\boldsymbol{s}_t\}$ is not stationary, we could choose some diffuse beliefs

$$\boldsymbol{s}_{0|0} = \boldsymbol{0}, \quad \boldsymbol{P}_{0|0} = \tau^2 \boldsymbol{I},$$

  with very large τ^2.

```
def kalman_filter(Y, c, G, V, F, U, s0, P0):
    T, n = Y.shape
    m = F.shape[0]

    s_pred = np.zeros((T, m))       # s_{t|t-1}
    P_pred = np.zeros((T, m, m))    # P_{t|t-1}
    s_filt = np.zeros((T, m))       # s_{t|t}
    P_filt = np.zeros((T, m, m))    # P_{t|t}
    y_pred = np.zeros((T, n))       # y_{t|t-1}
    Q_pred = np.zeros((T, n, n))    # Q_{t|t-1}

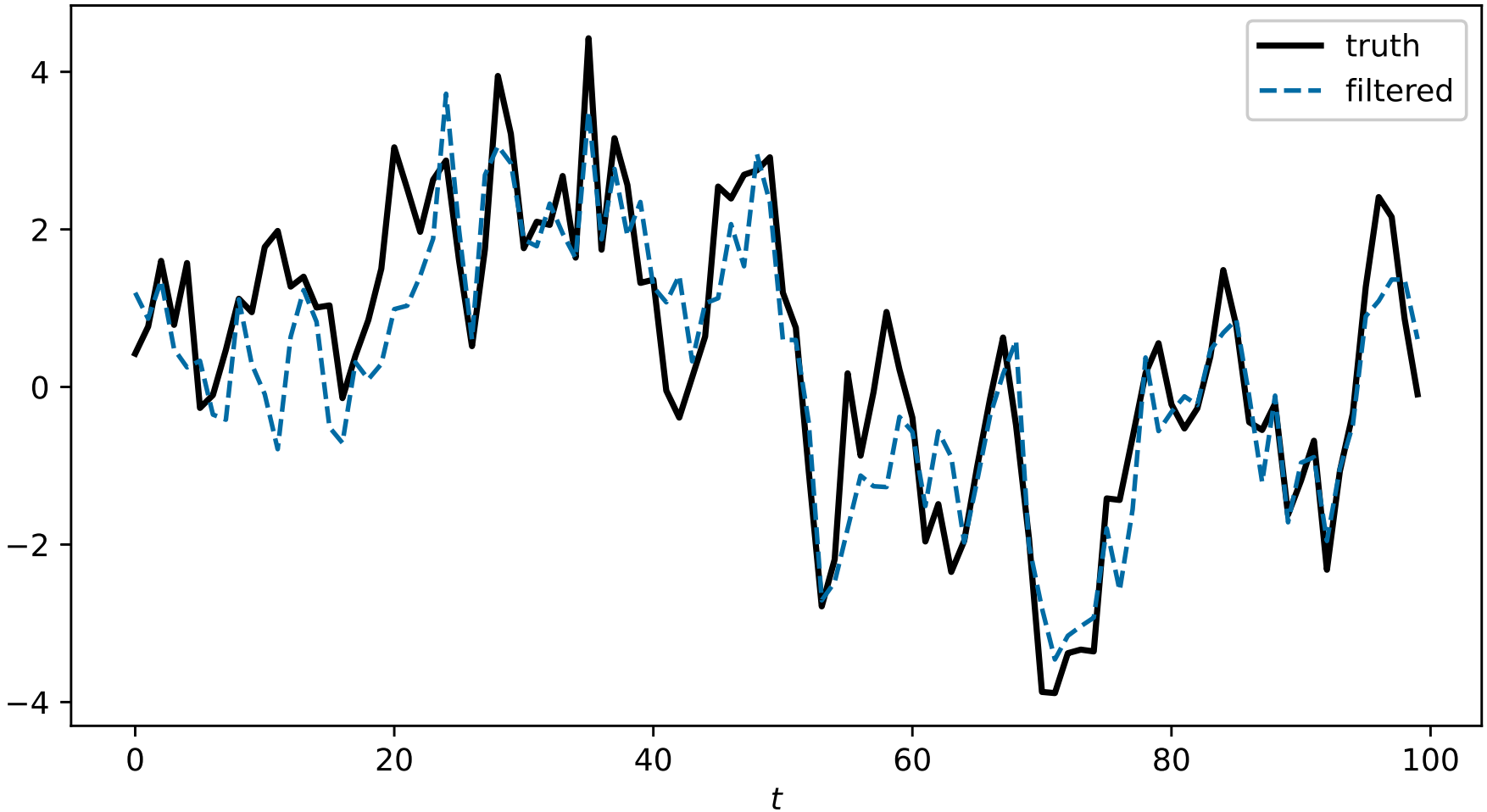
    s_prev, P_prev = s0, P0
    for t in range(T):
        # Predict
        s_pred[t] = F @ s_prev
        P_pred[t] = F @ P_prev @ F.T + U
        y_pred[t] = c + G @ s_pred[t]
        Q_pred[t] = G @ P_pred[t] @ G.T + V

        # Gain
        K = np.linalg.solve(Q_pred[t], G @ P_pred[t]).T

        # Update
        innov = Y[t] - y_pred[t]
        s_filt[t] = s_pred[t] + K @ innov
        P_filt[t] = P_pred[t] - K @ Q_pred[t] @ K.T

        s_prev, P_prev = s_filt[t], P_filt[t]

    return {'s_pred': s_pred, 'P_pred': P_pred,
            's_filt': s_filt, 'P_filt': P_filt,
            'y_pred': y_pred, 'Q_pred': Q_pred}
```



Kalman filter applied to the simulated data from earlier

Kalman smoother

- Kalman filter computes our "posterior" for $\boldsymbol{s}_t$ given data up to time t:

$$(\boldsymbol{s}_t \mid \{\boldsymbol{y}_s\}_{1 \leq s \leq t}) \sim N(\boldsymbol{s}_{t|t}, \boldsymbol{P}_{t|t}).$$

- But the data *after* time t may also be helpful for inferring $\boldsymbol{s}_t$.
- The **Kalman smoother** recursions (next slide) compute the full posterior:

$$(\boldsymbol{s}_t \mid \{\boldsymbol{y}_s\}_{1 \leq s \leq T}) \sim N(\boldsymbol{s}_{t|T}, \boldsymbol{P}_{t|T}).$$

- Derivation: Brockwell & Davis textbook, chapter 9.4.

Kalman smoother: algorithm

1. Run the Kalman filter to obtain $\{\boldsymbol{s}_{t|t-1}, \boldsymbol{P}_{t|t-1}, \boldsymbol{s}_{t|t}, \boldsymbol{P}_{t|t}\}_{t=1}^{T}$.
2. For each $t = T-1, T-2, \ldots, 1$ (going backwards in time):
   a) Compute $\boldsymbol{J}_t = \boldsymbol{P}_{t|t}\boldsymbol{F}'\boldsymbol{P}_{t+1|t}^{-1}$.
   b) Iterate:

$$\boldsymbol{s}_{t|T} = \boldsymbol{s}_{t|t} + \boldsymbol{J}_t(\boldsymbol{s}_{t+1|T} - \boldsymbol{s}_{t+1|t}),$$
$$\boldsymbol{P}_{t|T} = \boldsymbol{P}_{t|t} + \boldsymbol{J}_t(\boldsymbol{P}_{t+1|T} - \boldsymbol{P}_{t+1|t})\boldsymbol{J}_t'.$$

```
def kalman_smoother(filt, F):
    s_pred, P_pred = filt['s_pred'], filt['P_pred']
    s_filt, P_filt = filt['s_filt'], filt['P_filt']
    T, m = s_filt.shape

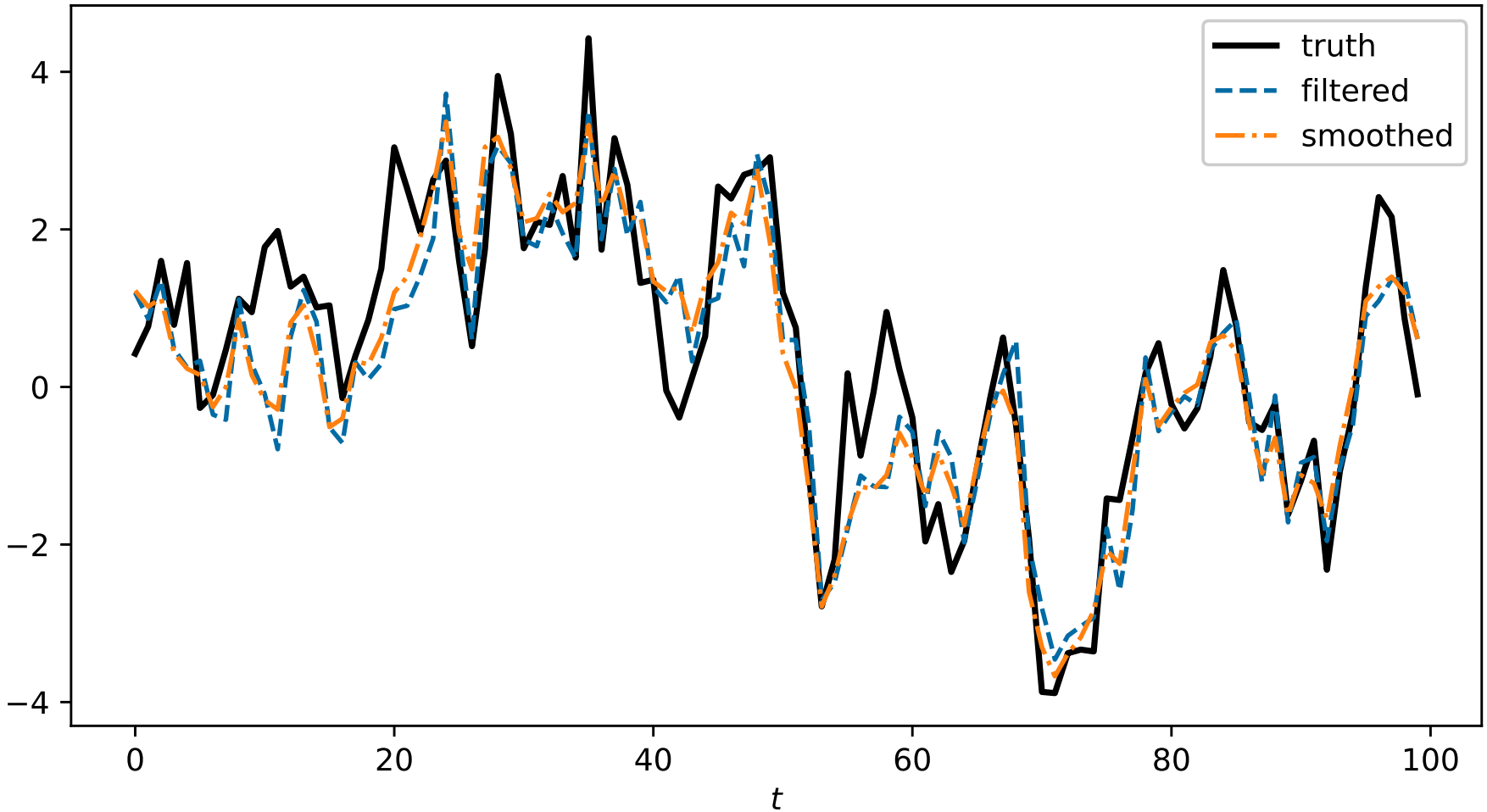
    s_smooth = np.zeros((T, m))     # s_{t|T}
    P_smooth = np.zeros((T, m, m))  # P_{t|T}

    # Initialize at final filtered state
    s_smooth[T - 1] = s_filt[T - 1]
    P_smooth[T - 1] = P_filt[T - 1]

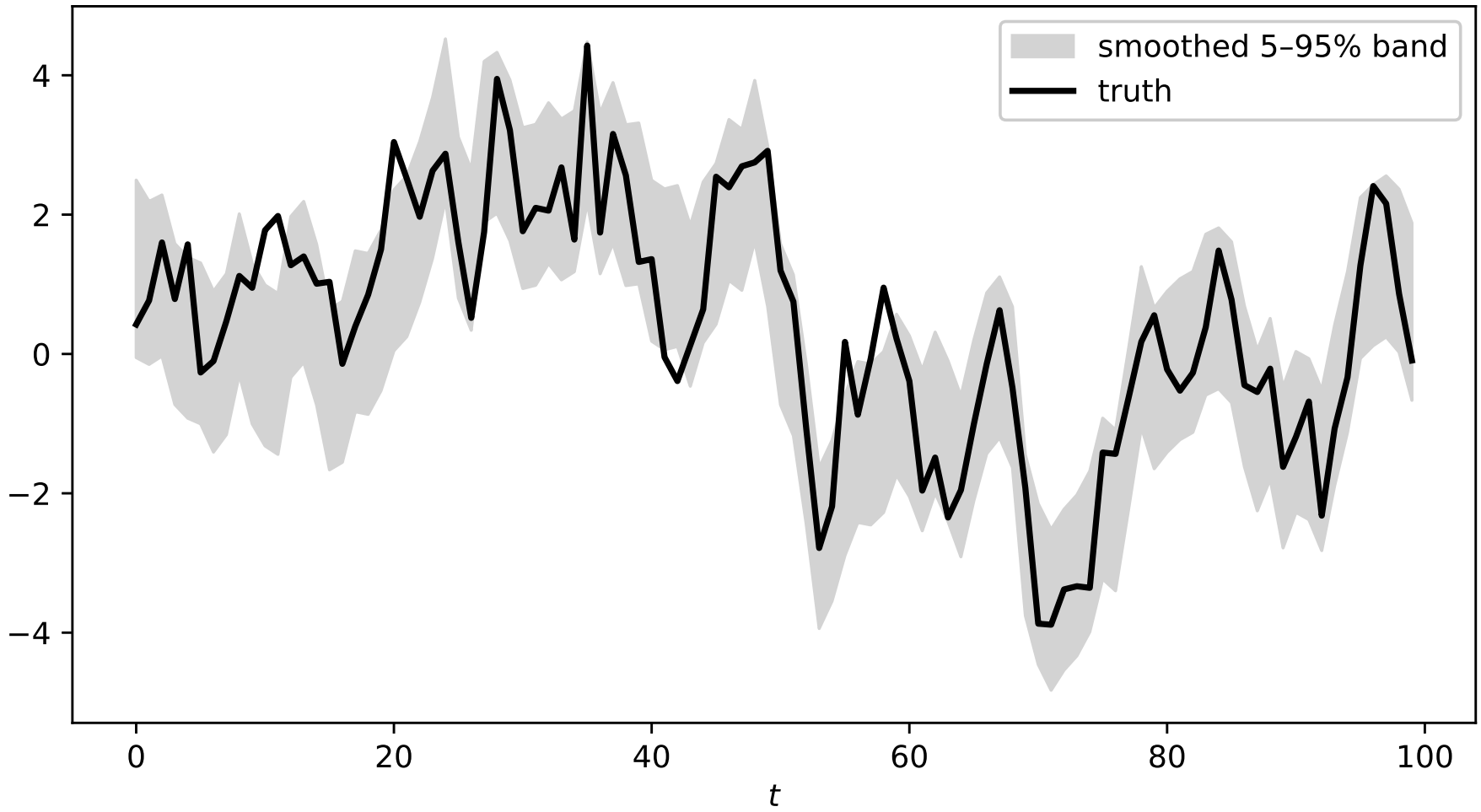
    for t in range(T - 2, -1, -1):
        J = np.linalg.solve(P_pred[t + 1], F @ P_filt[t]).T
        s_smooth[t] = s_filt[t] + J @ (s_smooth[t + 1] - s_pred[t + 1])
        P_smooth[t] = P_filt[t] + J @ (P_smooth[t + 1] - P_pred[t + 1]) @ J.T

    return {'s_smooth': s_smooth, 'P_smooth': P_smooth}
```

Note: you can also use the pre-packaged Kalman filter and smoother in `statsmodels.tsa.statespace`. Just make sure you understand the notation.



Kalman filter and smoother applied to the simulated data from earlier


smoothed 5–95% band
truth
4
2
0
−2
−4
0
20
40
60
80
100
t

Bayesian estimation of state space models

- So far we've assumed that the parameters of the state space model were known.
- Let's now consider Bayesian estimation of these parameters.
- Suppose we have formulated a prior on the underlying parameter vector $\boldsymbol{\theta}$. The coefficient matrices $\{\boldsymbol{c}, \boldsymbol{F}, \boldsymbol{G}, \boldsymbol{U}, \boldsymbol{V}\}$ are functions of $\boldsymbol{\theta}$.
- How to compute the likelihood?

Bayesian estimation: likelihood function

- From the Kalman filter we know

$$(\boldsymbol{y}_t \mid \{\boldsymbol{y}_s\}_{1\leq s\leq t-1}, \boldsymbol{\theta}) \sim N(\boldsymbol{y}_{t|t-1}, \boldsymbol{Q}_{t|t-1}).$$

  - $\boldsymbol{y}_{t|t-1}$ and $\boldsymbol{Q}_{t|t-1}$ are implicitly functions of $\boldsymbol{\theta}$.

- This gives us all necessary ingredients for the likelihood:

$$f(\boldsymbol{y}_1, \ldots, \boldsymbol{y}_T \mid \boldsymbol{\theta}) = \prod_{t=1}^{T} f(\boldsymbol{y}_t \mid \{\boldsymbol{y}_s\}_{1\leq s\leq t-1}, \boldsymbol{\theta})$$

$$= \prod_{t=1}^{T} (2\pi)^{-n/2} \det(\boldsymbol{Q}_{t|t-1})^{-1/2} e^{-\frac{1}{2}(\boldsymbol{y}_t - \boldsymbol{y}_{t|t-1})' \boldsymbol{Q}_{t|t-1}^{-1} (\boldsymbol{y}_t - \boldsymbol{y}_{t|t-1})}.$$

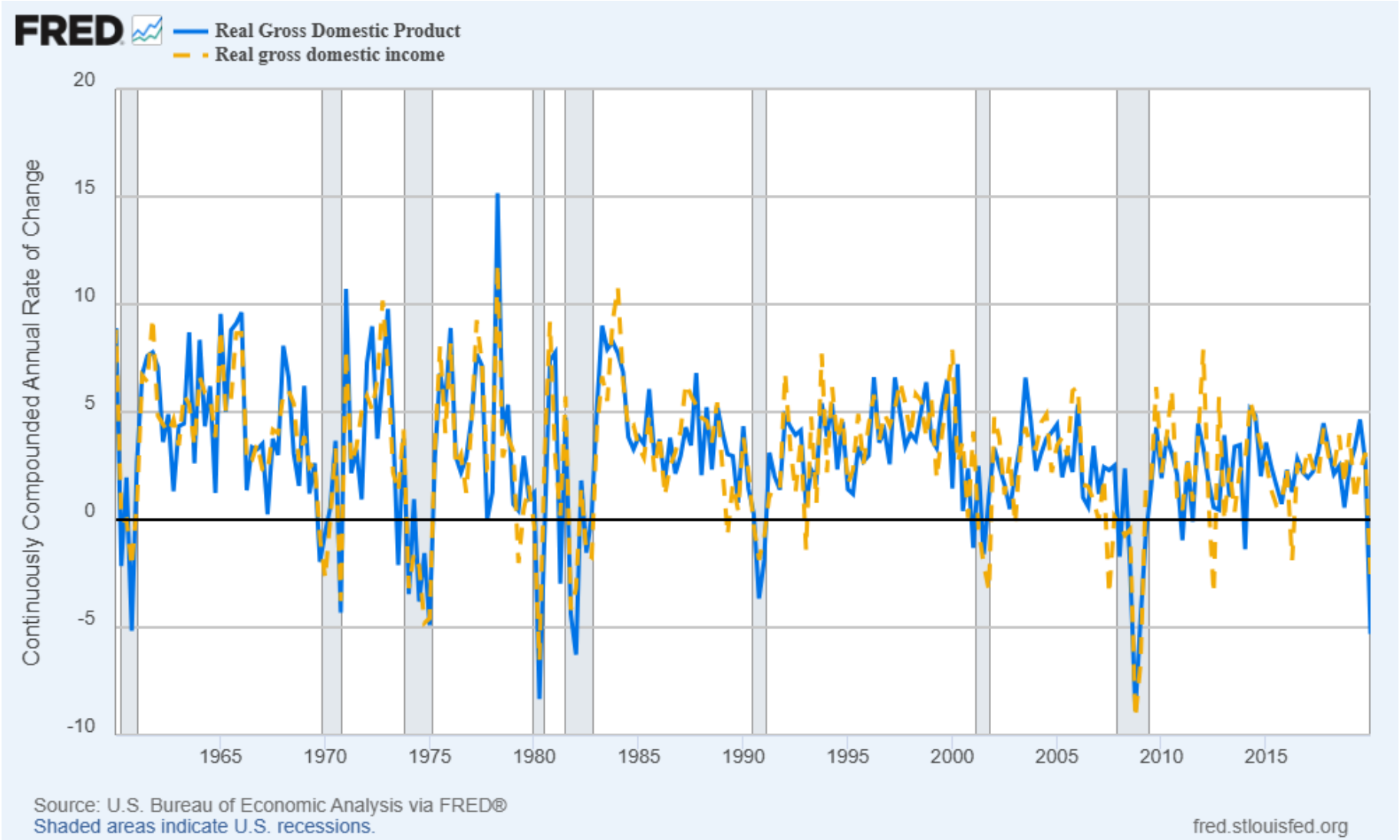
- We could use this to compute a **maximum likelihood estimator**

$$\hat{\boldsymbol{\theta}}^{\text{MLE}} \equiv \text{argmax}_{\boldsymbol{\theta}} \; f(\boldsymbol{y}_1, \ldots, \boldsymbol{y}_T \mid \boldsymbol{\theta})$$

via numerical optimization. But we will focus on Bayesian inference.

Application: GDP measurement

- The Bureau of Economic Analysis produces two measures of GDP that should theoretically be equivalent:
  1. Expenditure-side measure GDP_{Et}: sum of all expenditures on final goods and services.
  2. Income-side measure GDP_{It}: sum of all income generated in production (also called Gross Domestic Income, GDI).
- But in practice these two series can diverge significantly due to imperfections in data collection.


FRED
Real Gross Domestic Product
Real gross domestic income
Continuously Compounded Annual Rate of Change
20
15
10
5
0
-5
-10
1965
1970
1975
1980
1985
1990
1995
2000
2005
2010
2015
Source: U.S. Bureau of Economic Analysis via FRED®
Shaded areas indicate U.S. recessions.
fred.stlouisfed.org

	GDP	GDI	GDP - GDI
count	239.000000	239.000000	239.000000
mean	2.989405	2.980319	0.009085
std	3.235301	3.159994	1.953780
min	-8.853365	-9.192442	-6.430172
25%	1.397032	1.282381	-1.311526
50%	3.040811	3.235237	-0.009659
75%	4.519602	4.897933	1.228413
max	15.167842	11.874460	5.751909

Units: annualized log growth rates, real

Application: GDP measurement (cont.)

- We model the real GDP measurements as a dynamic factor model.
  - Simplest model in Aruoba, Diebold, Nalewaik, Schorfheide & Song (2016). See paper for more elaborate approaches.
- Assume latent true GDP growth $\tilde{s}_t$ evolves as an AR(1):

$$\tilde{s}_t = \tilde{c} + \rho \tilde{s}_{t-1} + u_t, \quad u_t \overset{iid}{\sim} N(0, \sigma_G^2).$$

- The two observed GDP measures are noisy proxies of true GDP:

$$\text{GDP}_{Et} = \tilde{s}_t + v_{Et}, \quad v_{Et} \overset{iid}{\sim} N(0, \sigma_E^2),$$
$$\text{GDP}_{It} = \tilde{s}_t + v_{It}, \quad v_{It} \overset{iid}{\sim} N(0, \sigma_I^2).$$

- Errors/shocks $\{v_{Et}\}$, $\{v_{It}\}$, and $\{u_t\}$ are independent of each other.

Application: GDP measurement (cont.)

- Define $s_t \equiv \tilde{s}_t - \mu$, where $\mu \equiv E[\tilde{s}_t] = \frac{\tilde{c}}{1-\rho}$.
- State space representation:

$$\boldsymbol{y}_t = \boldsymbol{c} + \boldsymbol{G}s_t + \boldsymbol{v}_t, \quad s_t = Fs_{t-1} + u_t,$$

$$\text{where} \quad \boldsymbol{y}_t \equiv \begin{pmatrix} \text{GDP}_{Et} \\ \text{GDP}_{It} \end{pmatrix}, \quad \boldsymbol{c} \equiv \begin{pmatrix} \mu \\ \mu \end{pmatrix}, \quad \boldsymbol{G} \equiv \begin{pmatrix} 1 \\ 1 \end{pmatrix},$$

$$F \equiv \rho, \quad \text{Var}(u_t) \equiv \sigma_G^2, \quad \text{Var}(\boldsymbol{v}_t) \equiv \begin{pmatrix} \sigma_E^2 & 0 \\ 0 & \sigma_I^2 \end{pmatrix}.$$

- Prior distribution (all parameters independent):

$$\mu \sim N(3, 100), \quad \rho \sim N(0.3, 1) \text{ truncated to } [-1, 1],$$

$$\sigma_j \sim \text{Cauchy}_+(0, 5),\ j \in \{E, I, G\}.$$

```
data {
    int<lower=1> T;             // sample size
    matrix[T, 2] y;             // data
    real mu_mean;               // prior mean for mu
    real<lower=0> mu_std;       // prior std. dev. for mu
    real rho_mean;              // prior mean for rho
    real<lower=0> rho_std;      // prior std. dev. for rho
    real<lower=0> sigma_scale;  // prior scale for sigma_j
}

transformed data {
    vector[2] G = [1.0, 1.0]';
}

parameters {
    real mu;
    real<lower=-1, upper=1> rho;
    vector<lower=0>[2] sigma_v;
    real<lower=0> sigma_G;
}

transformed parameters {
    vector[2] c = [mu, mu]';
    matrix[2, 2] V = diag_matrix(square(sigma_v));
}
```

(continued on next slide)

```
model {
    // Prior
    mu ~ normal(mu_mean, mu_std);
    rho ~ normal(rho_mean, rho_std);
    sigma_v ~ cauchy(0, sigma_scale);
    sigma_G ~ cauchy(0, sigma_scale);

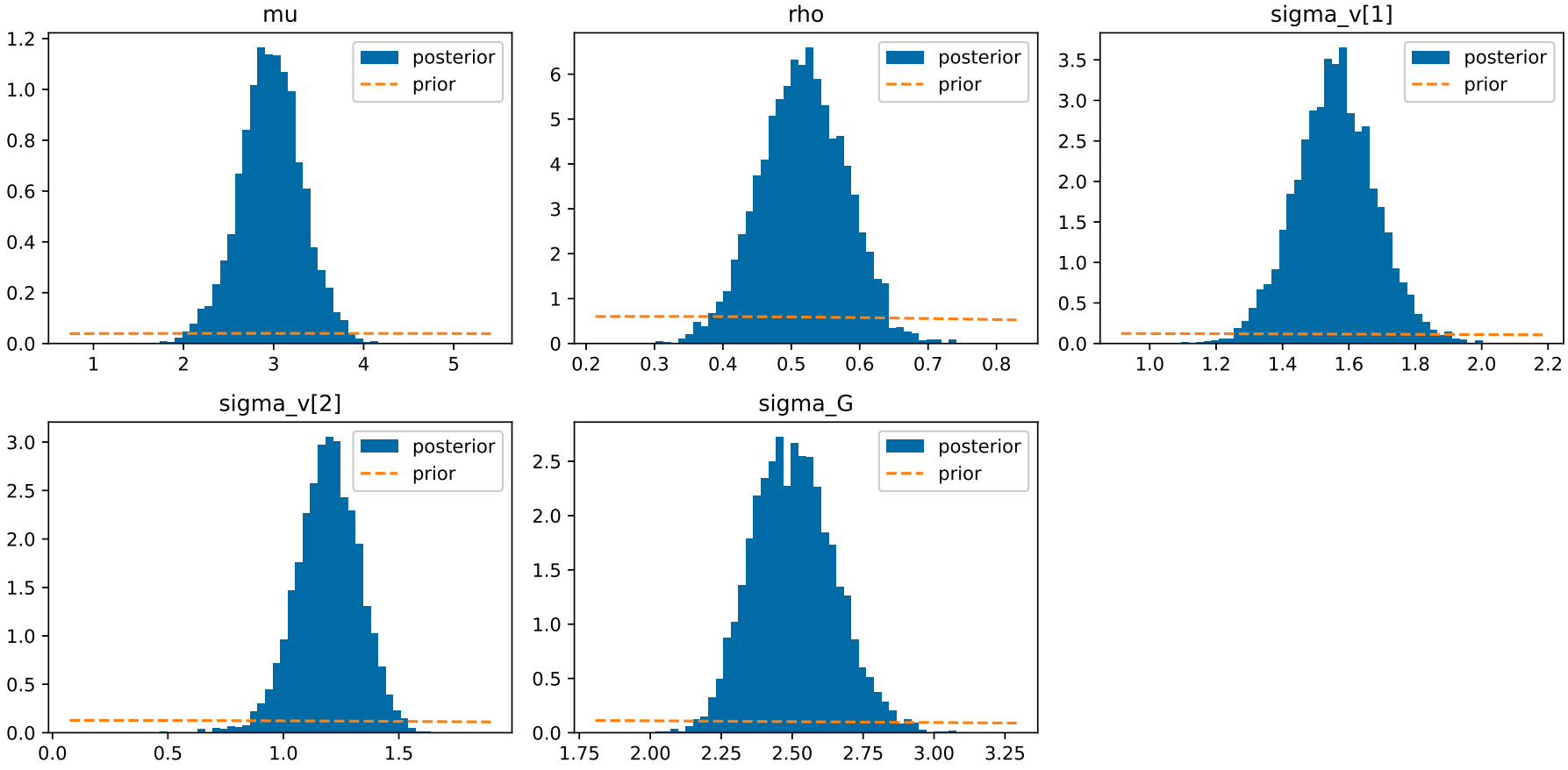
    // Likelihood via Kalman filter
    {
        // Initialize
        real s = 0;
        real P = square(sigma_G) / (1 - square(rho));

        for (t in 1:T) {
            // Predict
            real s_pred = rho * s;
            real P_pred = square(rho) * P + square(sigma_G);
            vector[2] y_pred = c + G * s_pred;
            matrix[2, 2] Q_pred = P_pred * G * G' + V;

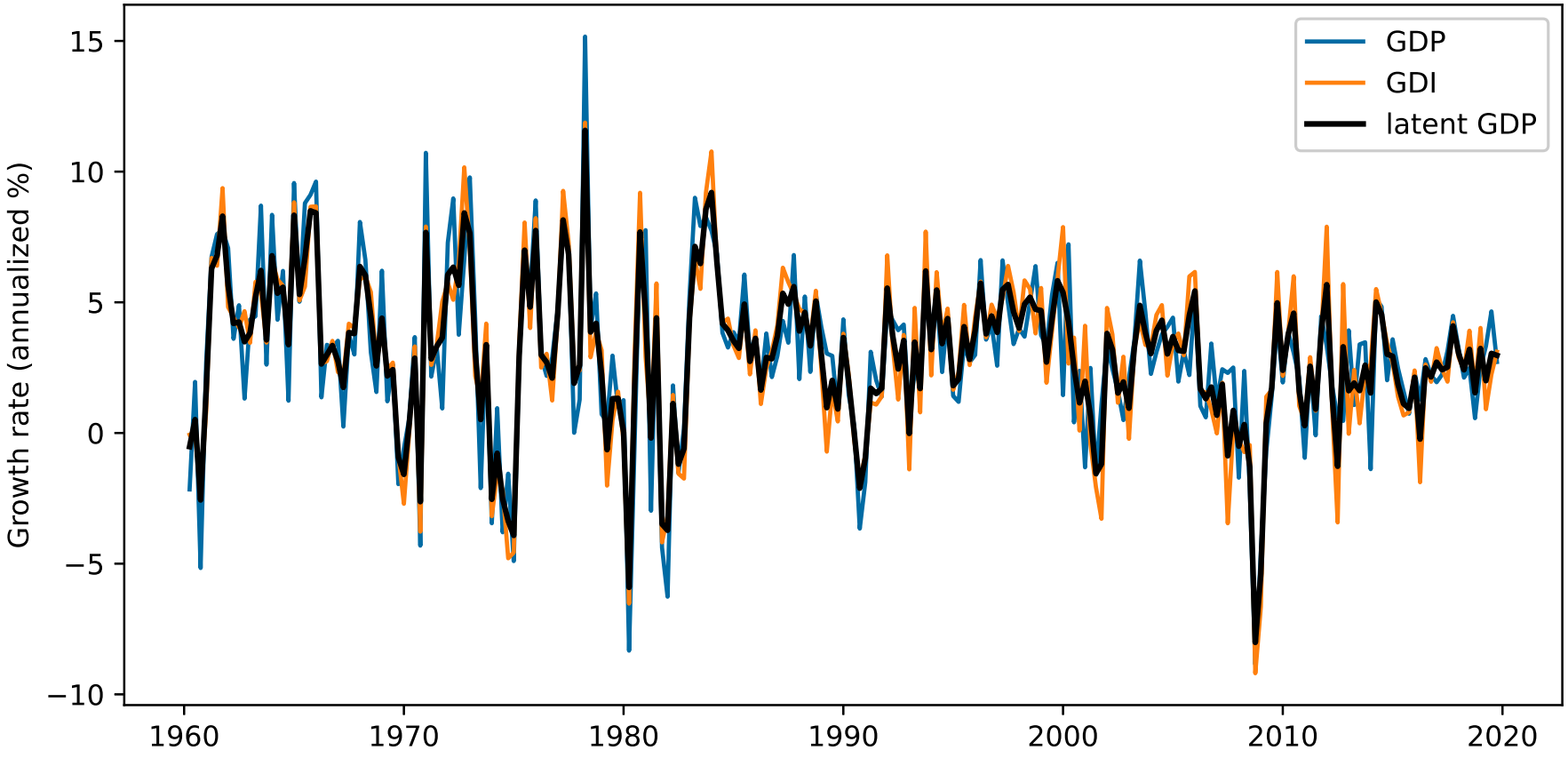
            // Likelihood contribution
            y[t]' ~ multi_normal(y_pred, Q_pred);

            // Gain (transposed)
            vector[2] Kt = mdivide_left_spd(Q_pred, G * P_pred);

            // Update
            s = s_pred + Kt' * (y[t]' - y_pred);
            P = P_pred - Kt' * Q_pred * Kt;
        }
    }
}
```



Note: $\texttt{sigma_v[1]} = \sigma_E$, $\texttt{sigma_v[2]} = \sigma_I$



Plot of $E[\tilde{s}_t \mid \boldsymbol{y}_1, \ldots, \boldsymbol{y}_T]$ across t

Computed by averaging $\mu + \underbrace{E(s_t \mid \boldsymbol{y}_1, \ldots, \boldsymbol{y}_T, \boldsymbol{\theta})}_{\text{Kalman smoother}}$ across parameter draws

What did we learn?

- State space models
  - Examples: dynamic factor model, ARMA
- Kalman filter and smoother
- Likelihood function $\Rightarrow$ Bayesian inference
- Application: GDP measurement

Looking ahead

- Modeling time-varying parameters
- Missing data
- Mixed-frequency data
- High-dimensional factor models
- Models with regime switching